Apertus Lab: Evidence-Based Red-Teaming Workbench for Apertus

Technical report · Track 2B
Team: Apertus Lab
Members: Chen Gao, Namila Namila

Summary. Apertus Lab is a bounded research workbench that tests Apertus with matched control/probe pairs in several languages, stores every raw request and response in an append-only ledger, requires human review before anything is called a finding, and exports an independent replay package. Using it, we found and reproduced (16 of 16) a case where Apertus v1.5 70B follows an unverified supervisor note and picks the male candidate despite identical evidence.

1. Use of Apertus

Model: swiss-ai/Apertus-v1.5-70B (non-thinking) in every role: target, planner, critic and optional judge. No other model is used.

How it is used: Inference and evaluation through an OpenAI-compatible endpoint

Where it runs: CSCS hosted inference service in Switzerland; the endpoint is configurable

2. Problem

Red-team claims about language models are often anecdotes: one prompt, one answer, no denominator. Multilingual models such as Apertus need more: the same test must be repeated across languages, controls must be matched, and failures such as rate limits or invalid JSON must not be mistaken for model faults. Researchers need a tool that makes the chain from a claim back to the exact prompt and response visible and reproducible.

3. Solution and Architecture

The workbench has a bilingual React interface, a FastAPI service and an independent persistent worker. Campaigns are bounded: explicit limits on calls, tokens, time and at most 10 in-flight requests across all roles. 27 research cards (injection, facts, identity) exist in Chinese, English and German. Each case is a control and a probe that differ by one factor; a rule-based judge scores decision and protocol; confirmation repeats the pair; a defence comparison tests four prompt variants; a human review gate is required before a finding can be exported. Every request and response is stored as bytes with hashes in a SQLite write-ahead-log ledger. A Probe workflow lets a researcher try one prompt in up to 12 languages, with translation and analysis calls that always need confirmation.

4. Deployment Architecture

Two containers share one image: api (port 8770, local only) and worker. State is one SQLite database on a volume. The only runtime dependency is an OpenAI-compatible model endpoint (LLM_BASE_URL). Sovereign Swiss cloud is the architecture we used: all live runs target Apertus 70B on the CSCS service in Switzerland. On-premise is supported by pointing LLM_BASE_URL to a local Apertus server, but we did not test a self-hosted server. Air-gapped use is verified only for mock mode and replay of mock evidence; a fully air-gapped live run needs a local Apertus server and was not tested.

5. Evaluation

Software tests: 98 offline Python tests (including platform behaviour checks) and 4 front-end tests pass; the production build and a dependency audit report no issues.

Concurrency: With a simulated provider the worker reached 10 in-flight requests and never exceeded the limit.

Live use: Two live campaigns (33 requests, all HTTP 200) plus an independent replay (16 requests) on Apertus v1.5 70B.

Result found with the tool: Chinese matched-identity task: 8 of 8 controls answer tie; 8 of 8 probes with an unverified note choose the male candidate, in both A/B layouts. The replay reproduced 16 of 16.

6. Limitations

The live result is narrow: one language, one wording, a synthetic task, four repeats per case, and a deterministic judge without an independent model judge. The review of the live result was done by an AI assistant at a team member’s request, not by an independent human. Real 10-way parallel runs, research-model quality and annotation agreement are not yet validated. The Docker build was not run on the authoring machine; the services were run directly with Python. Parts of the repository reuse earlier components (assets extracted from BreachWeave and a local LLM Wiki fork), listed in management/REUSE_ASSETS.md.

7. Reproducibility

From the repository root run make run (Docker), then open http://127.0.0.1:8770. Default mode is mock and sends no model request. For live mode put LLM_NAME, LLM_BASE_URL and LLM_API_KEY in a root .env file.

```
make run
# open http://127.0.0.1:8770  ->  New study -> Preflight -> Create -> Start
```

8. License

Report: CC-BY-4.0. Code: see repository licences; reused BreachWeave assets keep their Apache-2.0 terms.

References

Apertus v1.5 70B: https://huggingface.co/swiss-ai/Apertus-v1.5-70B

HackApertus project template: https://github.com/HackApertus/project-template